

Annex — Dataset Documentation

This annex documents the five datasets used in the Data Analyst Technical Case. It is reference material — read it once before you start writing SQL, then return to it as needed. The brief itself is in [Data_Analyst_Case.pdf](#).

Dataset 1 — Airline Operations Data

Three separate CSVs, one per airline, from three independent data providers. The schemas are **not pre-aligned** — column names, formats, and conventions differ across them. Unifying them is part of the case.

File	Airline	IATA	ICAO	Country of operations	Rows (~)
<code>ak_output_jun_aug_2025.csv</code>	AirAsia	AK	AXM	Southeast Asia, China, India, Australia	1,005
<code>jt_output_jun_aug_2025.csv</code>	Lion Air	JT	LNI	Indonesia (domestic + intl. to MY, KR, CN, SA)	1,108
<code>y4_output_jun_aug_2025.csv</code>	Volaris	Y4	VOI	Mexico, USA, Central / South America	1,106

Each file is grain-of-flight-leg (one row per flight leg). Coverage windows differ: **AK** is concentrated in the first week of June (Jun 1–6); **JT** spans Jun 1–14 with frequency tapering off; **Y4** has the broadest span, Jun 1 – Aug 15. Cross-airline comparisons over the full window are not apples-to-apples — be explicit about your analysis window.

Column mapping (combined view)

The table below is the practical mapping you need to build the unified schema. Where a field needs special handling — a JSON envelope, a UTC→local conversion, a quoted/padded value — that’s flagged in the notes column.

Concept	AK column	JT column	Y4 column	Notes
Carrier code	<code>flightCarrier</code>	<code>airlineCode</code>	<code>carrier</code>	JT values may be inconsistent.
Flight number	<code>flightNumber</code> (string, padded, quoted: "AK 889")	<code>flightNumber</code> (int)	<code>flightNumber</code> (int)	Strip whitespace and quotes; keep the numeric portion only.
Operational date	<code>flightDate</code> (YYYY-MM-DD)	parse from <code>timings</code> JSON	<code>departureDate</code> (typically YYYY-MM-DD)	Y4 has a small number of rows in other date formats.
Departure airport (IATA)	<code>flightDep</code>	<code>originAirport</code> JSON → <code>\$.code.S</code>	<code>departure</code>	JT requires JSON parsing — see “Working with the JSON columns” below.
Arrival airport (IATA)	<code>flightArr</code>	<code>destAirport</code> JSON → <code>\$.code.S</code>	<code>destination</code>	Same JSON note for JT.
Scheduled departure	<code>flight_dep_scheduled_full</code> (local)	<code>timings.scheduledDepTime</code> (epoch ms, UTC)	<code>departureDate</code> + <code>scheduledDeparture</code> (local)	JT is in UTC — convert to local using the timezone in <code>airports_master</code> .
Estimated departure	<code>flight_dep_estimated_full</code> (local; may be literal "null")	<code>timings.estimatedDepTime</code> (epoch ms, UTC)	<code>departureDate</code> + <code>estimatedDeparture</code> (local)	Watch for the literal string "null" in AK.
Actual departure	<code>flight_dep_actual_full</code> (local)	<code>timings.actualDepTime</code> (epoch ms, UTC)	not provided (use <code>delayDuration</code>)	Y4 doesn’t expose actual times directly; reconstruct from <code>delayDuration</code> if needed.
Scheduled arrival	<code>flight_arr_scheduled_full</code>	<code>timings.scheduledArrTime</code>	<code>arrivalDate</code> + <code>scheduledArrival</code>	Same UTC note for JT.
Estimated arrival	<code>flight_arr_estimated_full</code>	<code>timings.estimatedArrTime</code>	<code>arrivalDate</code> + <code>estimatedArrival</code>	
Actual arrival	<code>flight_arr_actual_full</code>	<code>timings.actualArrTime</code>	not provided	
Status	<code>flightStatus</code> (Landed /	<code>providerStatus</code> +	<code>flightStatus</code> (On	Status vocabulary

	Departed / On Time / Delayed / Diverted / Cancelled, casing varies)	statusMap JSON	Time / Delayed / Canceled — American spelling)	varies across and within files. Define a controlled vocabulary in Part 1.
Aircraft type	not provided directly (<code>rego</code> is tail number)	<code>aircraftType</code> (e.g. B739 , B738)	<code>aircraftFamily</code> (e.g. A320 , A321)	Best-available value across files.
Distance / duration	not in this extract	<code>distance</code> (km), <code>duration</code> (ms), <code>durationStr</code>	<code>duration</code> (HH:MM)	Reference only; not needed for required questions.

File-specific notes

`ak_output_jun_aug_2025.csv` (**AirAsia / AK**) — 61 columns; the widest of the three. Most operational columns are split into `*_date`, `*_time`, `*_full`, and `*_UTC` variants — the `*_full` columns already capture the local datetime, so the split variants are largely redundant. The file also contains `prev_*` columns describing the previous leg of each rotation, plus operational metadata (`boardGate`, `checkinTable`, `baggageID`, `recordLocator`) — sometimes padded or quoted. The composite `flight_id` follows the pattern `<airline><number><date><dep><arr>`, but the encoded date and route inside `flight_id` may be stale — **treat `flightDate` as authoritative**, not `flight_id`.

`jt_output_jun_aug_2025.csv` (**Lion Air / JT**) — 29 columns, but several of the most important ones (`destAirport`, `originAirport`, `departureDeviation`, `arrivalDeviation`, `terminalInfo`, `timings`, `statusMap`) are embedded JSON strings in a DynamoDB-style envelope. Each value is wrapped as `{"S": "..."}` , `{"N": "..."}` , or `{"BOOL": ...}` indicating String, Number, or Boolean type.

Working with the JSON columns: each is a stringified object. To get the IATA destination code, parse `destAirport` and read `$.code.S`. The `S` / `N` / `BOOL` wrappers indicate the type of the inner value.

```
destAirport = {"code":{"S":"UPG"}, "cityName":{"S":"Ujung Pandang"}, ...}
→ code = "UPG"
```

In Postgres: `(destAirport::jsonb -> 'code' ->> 'S')`. BigQuery, Snowflake, and DuckDB have equivalent JSON functions. SQL Server: `JSON_VALUE`. If your dialect doesn't support JSON natively, parse in a pre-processing step before loading.

The `timings` JSON object holds scheduled, estimated, and actual times for both departure and arrival as **epoch milliseconds in UTC** — convert to local using the timezone in `airports_master`. The provider's globally unique ID is `flightUniqueId` (e.g. `JT675BPNUPG04062025`); a recent on-time performance metric is in `onTimeAccuracy` (e.g. `93.33%`).

`y4_output_jun_aug_2025.csv` (**Volaris / Y4**) — 24 columns; the cleanest schema, with flat fields and human-readable times. `delayDuration` is the total delay in `HH:MM` format (`00:00` indicates on-time, format may vary slightly). `irop` flags irregular operations. **A non-trivial subset of rows are entirely blank in the operational fields** — these are placeholder rows from the source provider for flights that were never published. Decide and document how you handle them. `flightStatus` uses American spelling (`Canceled`, not `Cancelled`); other variants may also appear.

Known data quality areas

This is a real-world-style extract, not a curated demo. Expect formatting inconsistencies, casing inconsistencies, embedded JSON, stale identifiers, exact duplicates, and a small number of impossible or contradictory rows. Cataloguing and resolving these issues is part of the case (see the brief).

Dataset 2 — Airports Master

File: `airports_master.csv`. **Rows:** 193. **Grain:** one row per airport.

A reference table covering all commercial airports relevant to the operational data, plus a number of additional international hubs that are not referenced by any operational extract. The master is intentionally broader than the operational footprint — joins should be performed in the appropriate direction.

Column	Type	Description
<code>iata_code</code>	string(3)	IATA airport code (e.g. <code>KUL</code> , <code>MEX</code> , <code>CGK</code>). Primary key.
<code>icao_code</code>	string(4)	ICAO airport code (e.g. <code>WMKK</code> , <code>MMMX</code> , <code>WIII</code>).
<code>airport_name</code>	string	Official airport name.
<code>city</code>	string	Primary city served.
<code>country</code>	string	Country name (English).
<code>country_code</code>	string(2)	ISO 3166-1 alpha-2 country code.
<code>latitude</code>	float	Decimal degrees.
<code>longitude</code>	float	Decimal degrees.
<code>timezone</code>	string	IANA timezone (e.g. <code>Asia/Kuala_Lumpur</code>).

178 airports are referenced by at least one operational record. 15 additional airports are global hubs included for reference only and are not referenced by the operational data in this extract (e.g. `LHR`, `CDG`, `FRA`, `NRT`, `HND`, `DXB`, `DOH`, `JNB`, `SYD`, `AKL`, `YYZ`, `YVR`, `GRU`, `EZE`, `SCL`).

Notes. `iata_code` is the natural join key against the operational data. The Lion Air extract embeds the IATA inside its JSON `originAirport` / `destAirport` columns under `$.code.S`. Coordinates are accurate to roughly five decimal places — suitable for haversine distance and timezone normalisation, not for navigation.

Dataset 3 — Ticket Prices by Route

File: `ticket_prices.csv`. **Rows:** ~1,299. **Grain:** one row per (airline, route, cabin class, validity period).

Reference pricing for one-way tickets. The basis for any revenue / yield estimation in the case.

Column	Type	Description
<code>route_id</code>	string	Surrogate row identifier (e.g. <code>R10001</code>). Not meaningful beyond row uniqueness.
<code>airline_icao</code>	string(3)	Airline ICAO code (<code>AXM</code> , <code>LNI</code> , <code>VOI</code> , etc.). Note: this is ICAO — operational data uses IATA. A lookup is required.
<code>dep_airport_iata</code>	string(3)	Departure airport IATA.
<code>arr_airport_iata</code>	string(3)	Arrival airport IATA.
<code>cabin_class</code>	string	<code>Economy</code> , <code>Business</code> , or (rarely) <code>First</code> .
<code>base_price_usd</code>	numeric	Base fare in USD (excluding taxes / fees).
<code>taxes_usd</code>	numeric	Taxes and fees in USD.
<code>total_price_usd</code>	numeric	<code>base_price_usd + taxes_usd</code> . Use this column when computing revenue.
<code>currency</code>	string	Always <code>USD</code> in this extract.
<code>effective_from</code>	date	Start of validity window.
<code>effective_to</code>	date	End of validity window.
<code>fare_code</code>	string	Provider-side fare code. Not used for joining.

Airline IATA ↔ ICAO mapping

Airline	IATA	ICAO
AirAsia	AK	AXM
Lion Air	JT	LNI
Volaris	Y4	VOI
Delta Air Lines	DL	DAL
American Airlines	AA	AAL
EVA Air	BR	EVA

Only the first three are referenced by the operational data. The other three appear as additional pricing rows that you will encounter and should handle correctly.

Coverage and gaps

- Pricing is provided for approximately **85% of unique routes** observed in the operational data, per airline. The remaining ~15% have no published fare — your queries must handle this gracefully (appropriate join type, documented decision on how to treat unpriced routes).
- Each priced route typically has two rows (Economy and Business). A small number also have a `First` row.
- A subset of rows have an `effective_from` of `2025-09-01` — future pricing not applicable to the analysis window. Filter accordingly.
- The dataset includes priced routes for airlines (`DAL`, `AAL`, `EVA`) and routes (e.g. `JFK-LHR`, `LAX-NRT`) that do not appear in the operational data. These are reference rows and should not contaminate operational metrics.

Notes on direction and aggregation

- Routes are **directional**: `(KUL, SIN)` and `(SIN, KUL)` are two separate rows with potentially different prices. Match on the directional pair.
- Because most routes have multiple cabin-class rows, naive aggregation (e.g. `SUM(total_price_usd)` across the table) will double- or triple-count routes. When estimating per-flight revenue, be explicit about which fare you're using (e.g. weighted average across cabins, or Economy-only as a baseline) and document the assumption.